

[Go to Questions](#) ↓

Let's use the `mov` instruction as the very first instructions in our module project `fibonacci`. We will need to load the initial values ($F_0=0$ and $F_1=1$) to `rax` and `rbx`, such that `rax = 0` and

`rbx = 1`. Let us copy the following code to a `fib.s` file:

```
global _start

section .text
_start:
    mov rax, 0
    mov rbx, 1
```

nasm

Now, let's assemble this code and run it with `gdb` to see how the `mov` instruction works in action:

```
$ ./assembler.sh fib.s -g
gef> b _start
Breakpoint 1 at 0x401000
gef> r
```

```
code:x86:64 ———
→ 0x401000 <_start+0>      mov    eax, 0x0
   0x401005 <_start+5>      mov    ebx, 0x1
```

```
registers ——— $rax    : 0x0
$rbx          : 0x0
```

...SNIP...

```
code:x86:64 ———
      0x401000 <_start+0>      mov    eax, 0x0
→ 0x401005 <_start+5>      mov    ebx,
0x1
registers ——— $rax    : 0x0
$rbx          : 0x1
```

Like this, we have loaded the initial values into our registers to later perform other operations and instructions on them.

Note: In assembly, moving data does not affect the source operand. So, we can consider `mov` as a `copy` function, rather than an actual move.

Loading Data

We can load immediate data using the `mov` instruction. For example, we can load the value of `1` into the `rax` register using the `mov rax, 1` instruction. We have to remember here that `the size of the loaded data depends on the size of the destination register`. For example, in the above `mov rax, 1` instruction, since we used the 64-bit register `rax`, it will be moving a 64-bit representation of the number `1` (i.e. `0x00000001`), which is not very efficient.

`This is why it is more efficient to use a register size that matches our data size`. For example, we will get the same result as the above example if we use `mov al, 1`, since we are moving 1-byte (`0x01`) into a 1-byte register (`al`), which is much more efficient. This is evident when we look at the disassembly of both instructions in `objdump`.

Let us take the following basic assembly code to compare the disassembly of both instructions:

```
global _start

section .text
_start:
    mov rax, 0
    mov rbx, 1
    mov bl, 1
```

nasm

Now let's assemble it and view its shellcode with `objdump` :

```
shellsession
kimvutrinh@htb[/htb]$ nasm -f elf64 fib.s && objdump -M intel -d fib.o
...SNIP...
0000000000000000 <_start>:
   0:  b8 00 00 00 00      mov     eax,0x0
   5:  bb 01 00 00 00      mov     ebx,0x1
  a:  b3 01              mov     bl,0x1
```

We can see that the shellcode of the first instruction is more than double the size of the last instruction.

This understanding will become very handy when writing shellcodes.

Let us modify our code to use sub-registers to make it more efficient:

```
nasm
global _start

section .text
_start:
    mov al, 0
    mov bl, 1
```

The `xchg` instruction will swap the data between the two registers. Try adding `xchg rax, rbx` to the end of the code, assemble it, and then run it through `gdb` to see how it works.

Address Pointers

Another critical concept to understand is using pointers. In many cases, we would see that the register or address we are using does not immediately contain the final value but contains another address that points to the final value. This is always the case with *pointer* registers, like `rsp`, `rbp`, and `rip`, but is also used with any other register or memory address.

For example, let's assemble and run `gdb` on our assembled `fib` binary, and check the `rsp` and `rip` registers:

```
gdb -q ./fib
gef> b _start
Breakpoint 1 at 0x401000
gef> r
...SNIP...
$rsp : 0x00007fffffffef490 → 0x0000000000000001
$rip : 0x0000000000401000 → <_start+0> mov eax, 0x0
```

We see that both registers contain pointer addresses to other locations. `GEF` does an excellent job of showing us the final destination value.

Moving Pointer Values

We can see that the `rsp` register holds the final value of `0x1`, and its immediate value is a pointer address to `0x1`. So, if we were to use `mov rax, rsp`, we won't be moving the value `0x1` to `rax`, but we will be moving the pointer address `0x00007fffffffef490` to `rax`.

To move the actual value, we will have to use square brackets `[]`, which in `x86_64` assembly and `Intel` syntax means `load value at address`. So, in the same above example, if we wanted to move the final value `rsp` is pointing to, we can wrap `rsp` in square brackets, like `mov`

`rax, [rsp]` , and this `mov` instruction will move the final value rather than the immediate value (which is an address to the final value).

We can use square brackets to compute an address offset relative to a register or another address. For example, we can do `mov rax, [rsp+10]` to move the value stored 10 address away from `rsp` .

To properly demonstrate this, let us take the following example code:

```
global _start

section .text
_start:
    mov rax, rsp
    mov rax, [rsp]
```

nasm

This is just a simple program to demonstrate this point and see the difference between the two instructions.

Now, let's assemble the code and run the program with gdb:

```
$ ./assembler.sh rsp.s -g
gef> b _start
Breakpoint 1 at 0x401000
gef> r
...SNIP...

code:x86:64
→ 0x401000 <_start+0>      mov    rax,
rsp
```

```
registers — $rax : 0x00007fffffffe490 → 0x0000000000000001
$rsp : 0x00007fffffffe490 → 0x0000000000000001
```

As we can see, the `mov rax, rsp` moved the immediate value stored at `rsp` (which is a pointer address to `rsp`) to the `rax` register. Now let's press `si` and check how `rax` will look after the second instruction:

```
$ ./assembler.sh rsp.s -g
gef> b _start
Breakpoint 1 at 0x401000
gef> r
...SNIP...

code:x86:64 —
→ 0x401003 <_start+3>      mov    rax, QWORD PTR
[rsp]
registers — $rax : 0x1
$rsp : 0x00007fffffffe490 → 0x0000000000000001
```

We can see that this time, the final value of `0x1` was moved into the `rax` register.

Note: When using `[]`, we may **need to set the data size** before the square brackets, like `byte` or `qword`. However, in most cases, `nasm` will **automatically** do that for us. We can see above that the final instruction is actually `mov rax, QWORD PTR [rsp]`. We also see that `nasm` also added `PTR` to specify moving a value from a pointer.

Loading Value Pointers

Finally, we need to understand how to load a pointer address to a value, using the `lea` (or `Load Effective Address`) instruction, which loads a pointer to the specified value, as in `lea rax, [rsp]`. This is the opposite of what we just learned above (i.e., load pointer to a value vs. move value from pointer).

In some instances, we need to load the address of a value to a certain register rather than directly load the value in that register. This is usually done when the data is large and would not fit in one register, so the data is placed on the stack or in the heap, and a pointer to its location is stored in the register.

For example, the `write` syscall we used in our `HelloWorld` program requires a pointer to the text to be printed, instead of directly providing the text, which may not fit in its entirety in the register, as the register is only 64-bits or 8 bytes.

First, if we wanted to load a direct pointer to a variable or a label, we can still use `mov` instructions. Since the variable name is a pointer to where it is located in memory, `mov` will store this pointer to the destination address. For example, both `mov rax, rsp` and `lea rax, [rsp]` will do the same thing of storing the pointer to `message` at `rax`.

However, if we wanted to **load a pointer with an offset** (i.e., a few addresses away from a variable or an address), we should use `lea`. This is why with `lea` the source operand is usually a variable, a label, or an address wrapped in square brackets, as in `lea rax, [rsp+10]`. This enables using offsets (i.e., `[rsp+10]`).

Note that if we use `mov rax, [rsp+10]`, it will actually move the value at `[rsp+10]` to `rax`, as discussed earlier. We cannot move a pointer with an offset using `mov`.

Let's take the following example to demonstrate how `lea` works and how it can differ from `mov`:

```

global _start

section .text
_start:
    lea rax, [rsp+10]
    mov rax, [rsp+10]

```

Now let's assemble it and run it with `gdb` :

```

$ ./assembler.sh lea.s -g
gef> b _start
Breakpoint 1 at 0x401000
gef> r
...SNIP...

code:x86:64 ———
→ 0x401003 <_start+0>      lea    rax,
[rsp+0xa]
registers ——— $rax : 0x00007fffffffe49a → 0x000000007ffffff
$rsp  : 0x00007fffffffe490 → 0x0000000000000001

```

We see that `lea rax, [rsp+10]` loaded the address that is 10 addresses away from `rsp` (in other words, 10 addresses away from top of stack). Now let's `si` to see what `mov rax, [rsp+10]` would do:

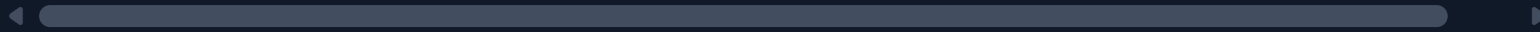
```

code:x86:64 ———
→ 0x401008 <_start+8>      mov    rax, QWORD PTR
[rsp+0xa]

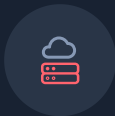
```

```
registers — $rax : 0x7fffffff  
$rsp : 0x00007fffffff490 → 0x0000000000000001
```

As expected, we see that `mov rax, [rsp+10]` moved the value stored there to `rax`.



Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline



Enable step-by-step solutions ⓘ PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1



XP +40



Add an instruction at the end of the attached code to move the value in "rsp" to "rax". What is the hex value of "rax" at the end of program execution?

Write your answer

Submit



Hint



Download File

← Previous

Section 11 / 24

 +20

Next

Mark Complete & Next